

Internship report

Alexandre Vannereux

1 Introduction

The internship was with Martin Pépin and Matthieu Dien as internship supervisors. The goal was to code and integrate pointing to the python library Usainboltz. This library implements the Boltzmann method which allows fast uniform sampling on combinatorial class at a fixed size [1].

2 Background

Before explaining what I did during the internship, the basic of combinatorial theory and usainboltz must be established.

2.1 Combinatorial theory

Definition 1 A class of combinatorial structures is a pair $\mathcal{C} = \langle \mathcal{C}, |\cdot|_{\mathcal{C}} \rangle$, where $\mathcal{C}$ is a finite or denumerable set and $|\cdot|_{\mathcal{C}}$ is an application from $\mathcal{C}$ to $\mathbb{N}$.

For $c \in \mathcal{C}$, $|c|_{\mathcal{C}}$ is the size of c . There must always be a finite number of elements of each size in $\mathcal{C}$.

The counting sequence $(C_n)_{n \in \mathbb{N}}$ associated with $\mathcal{C}$ is the number of elements of each size in $\mathcal{C}$. I.e. $\forall n \in \mathbb{N}, C_n = |\{c \in \mathcal{C} | |c|_{\mathcal{C}} = n\}|$.

Later on, we will indistinctively use $\mathcal{C}$ or $\langle \mathcal{C}, |\cdot| \rangle$. We will sometimes simplify the notation for the size: $|\cdot|$. When different class of combinatorial structures will be involved context can be used to differentiate their different size functions. Later on, we will indistinctively use $\mathcal{C}$ or $\langle \mathcal{C}, |\cdot| \rangle$. We will sometimes simplify the notation for the size: $|\cdot|$. When different class of combinatorial structures will be involved context can be used to differentiate their different size functions.

Definition 2 (Isomorphism) Two combinatorial class are said to be isomorphic if they have the same number of elements of each size.

We now introduce the two most frequently used and simple class of combinatorial structures. They are often used as base class to build more complex ones.

Definition 3 (Epsilon and atom) The epsilon class $\epsilon = \langle \mathcal{E}, |\cdot|_{\epsilon} \rangle$ is the class composed of only one element of size zero: $\mathcal{E} = \{e\}$ and $|e|_{\epsilon} = 0$.

Very similar, the atom class is comprised of exactly one element of size one: $\mathcal{Z} = \langle \{z\}, |\cdot|_{\mathcal{Z}} \rangle$ with $|z|_{\mathcal{Z}} = 1$.

We now define a number of operations on those objects:

Definition 4 (Product) Given two class of combinatorial structures, $\mathcal{A} = \langle \mathcal{A}, |\cdot|_{\mathcal{A}} \rangle$, $\mathcal{B} = \langle \mathcal{B}, |\cdot|_{\mathcal{B}} \rangle$, their product noted $\mathcal{A} \times \mathcal{B}$ is the class $\langle \mathcal{A} \times \mathcal{B}, |\cdot|_{\mathcal{A} \times \mathcal{B}} \rangle$ such that $\forall (a, b) \in \mathcal{A} \times \mathcal{B}, |(a, b)|_{\mathcal{A} \times \mathcal{B}} = |a|_{\mathcal{A}} + |b|_{\mathcal{B}}$.

As mentionned earlier, we can simplify the notations for the size function: $|\cdot|$ and referring as a class by its set: $\mathcal{C}$.

Definition 5 (Union) Given two class of combinatorial structures, $\mathcal{A}, \mathcal{B}$, we define their union $\mathcal{A} + \mathcal{B}$ as follows. Let μ and μ' such that $\mu \neq \mu'$ and $|\mu| = |\mu'| = 0$, then $\mathcal{A} + \mathcal{B} = (\{\mu\} \times \mathcal{A}) \cup (\{\mu'\} \times \mathcal{B})$. This is equivalent to changing elements of $\mathcal{B}$ such that $\mathcal{A}$ and $\mathcal{B}$ are disjoint.

This definition can be extended to the union of a finite or denumerable number of class of combinatorial structures as long as only a finite number of elements of size n appear in the union. For instance:

Definition 6 (Sequence) The sequence construction noted $\mathcal{C} = \text{Seq}(\mathcal{A})$ is defined as $\mathcal{C} = \mathcal{E} + \mathcal{A} + \mathcal{A} \times \mathcal{A} + \mathcal{A} \times \mathcal{A} \times \mathcal{A} + \dots$. To ensure that $\mathcal{C}$ is properly defined, $\mathcal{A}$ must not contain any element of size zero. Otherwise, there would be an infinite number of elements of same size.

Definition 7 (Labelled Set) The set construction noted $\mathcal{C} = \text{Set}(\mathcal{A})$ is the class of sets elements of $\mathcal{A}$. It can be seen as the quotient set of $\text{Seq}(\mathcal{A})$ by the relation $A = (A_1, \dots, A_n) \sim B = (B_1, \dots, B_n)$ if and only if B is a permutation of A .

Definition 8 (Labelled Cycle) The cycle construction noted $\mathcal{C} = \text{Cyc}(\mathcal{A})$ is the class of cycles of elements of $\mathcal{A}$. It can be seen as the quotient set of $\text{Seq}(\mathcal{A})$ by the relation $A = (A_1, \dots, A_n) \sim B = (B_1, \dots, B_n)$ if and only if B is a cyclic permutation of A .

Those three constructions have equivalent definitions with bounds on their number of elements. For instance $\text{Seq}(\mathcal{A}, \text{geq} = 5)$ means that we consider sequences with at least 5 elements of the class $\mathcal{A}$.

Using only products, unions, sequences, sets, cycles and finite sets to build class offer a very limited range of possibilities close to the expression level of regular languages. We can use recursive definition to obtain a range closer to context-free grammar.

Definition 9 (specification) A specification of a class of combinatorial structures $\mathcal{C}$ is a set of (possibly recursive) equations using only operations described above, atoms and epsilons such that $\mathcal{C}$ is the unique solution to the primary equation (up to an isomorphism).

$\mathcal{C}$ is said to be specifiable if it admits a specification of finite size.

This definition means that for any specifiable class $\mathcal{C}$, an element $c \in \mathcal{C}$ is composed of n atoms. Due to above definitions, each of these atoms is distinct from the other.

An alternate definition for sequences is by giving it a specification: $\mathcal{C} = \text{Seq}(\mathcal{A})$ is equivalent to $\mathcal{C}$ is solution of $\mathcal{C} = \mathcal{E} + \mathcal{A} \times \mathcal{C}$. Then the specification of $\mathcal{C}$ would be this equation joined with the specification of $\mathcal{A}$.

Now that we have combinatorial class and the capacity to build them, we can define an important tool to study them.

Definition 10 (ordinary and exponential generating functions) *Let $\mathcal{C}$ be an unlabelled combinatorial class, and (C_n) its counting sequence. The Ordinary Generating Function (OGF) of $\mathcal{C}$ is $C(z) = \sum_{n \in \mathbb{N}} z^n C_n$.*

Let $\mathcal{C}$ be a labelled combinatorial class, and (C_n) its counting sequence. The Exponential Generating Function (EGF) of $\mathcal{C}$ is $C(z) = \sum_{n \in \mathbb{N}} \frac{z^n C_n}{n!}$.

In both cases, we denote by ρ_C its radius of convergence.

A notation often used to obtain C_n from C is $[z^n]C(z) = C_n$.

The letter used for a combinatorial class will always be the same one used for its corresponding generating function.

Finally, we will often alternate between seeing C as a formal series or a power series. Context can be used to differentiate the two.

Property 1 (product, union and sequence on OGF) *Let $\mathcal{A}$ and $\mathcal{B}$ be two combinatorial class (labelled or unlabelled).*

- *If $\mathcal{C} = \mathcal{A} \times \mathcal{B}$ then $C(z) = A(z)B(z)$.*
- *If $\mathcal{C} = \mathcal{A} + \mathcal{B}$ then $C(z) = A(z) + B(z)$.*
- *If A has no objects of size 0 and $\mathcal{C} = \text{Seq}(\mathcal{A})$, then $C(z) = \frac{1}{1-A(z)}$.*
- *If A has no objects of size 0 and $\mathcal{C} = \text{Set}(\mathcal{A})$, then $C(z) = \exp(A(z))$.*
- *If A has no objects of size 0 and $\mathcal{C} = \text{Cyc}(\mathcal{A})$, then $C(z) = \ln(\frac{1}{1-A(z)})$.*

2.2 The library Usainboltz (UNiform SAMPLING with BOLTZmann)

3 Pointing

3.1 Grammar pointing

#Some shortcuts are allowed: $\text{None}-1 = \text{None}$; $(i-j) = \max(0, i-j)$. Very useful for sizes constraints.

```
def Point(S: SubRightRule) -> SubRightRule:
  switch S:
```

```

case Atom():
    return Atom()
case Epsilon() | Zero():
    return Zero()
case Union(R1, ..., Rn):
    return Union(Point(R1), ..., Point(Rn))
case Product(R1, ..., Rn):
    return Union(
        Product(Point(R1), R2, ..., Rn),
        ...,
        Product(R1, ..., Point(Rn))
    )
case RuleName(name):
    return RuleName(name + "_P")
case Set(R, leq = i, geq = j):
    return Product(R, Set(Point(R), leq = i-1, geq = j-1))
case Cycle(R, leq = i, geq = j):
    return Product(R, Seq(Point(R), leq = i-1, geq = j-1))
case Seq(R, leq = None, geq = None):
    return Union(Product(Seq(R, eq = 0), Point(R), Seq(R)))
case Seq(R, leq = i, geq = j):
    #with i != None.
    return Union(
        Product(Seq(R, eq = 0), Point(R), Seq(R, leq = i-1, geq = j-1)),
        Product(Seq(R, eq = 1), Point(R), Seq(R, leq = i-2, geq = j-2)),
        ...,
        Product(Seq(R, eq = i-1), Point(R), Seq(R, leq = 0, geq = j-i))
    )
case Seq(R, leq = None, geq = j):
    #with j != None
    return Union(
        Product(Seq(R, eq = 0), Point(R), Seq(R, leq = None, geq = j-1)),
        ...,
        Product(Seq(R, eq = j-2), Point(R), Seq(R, leq = None, geq = 1)),
        Product(Seq(R, geq = j-1), Point(R), Seq(R))
    )

```

3.2 Builder Pointing

4 Cycle Pointing

References

- [1] Flajolet P., Duchon P., Louchard G., and Schaeffer G. Boltzmann samplers for the random generation of combinatorial structures. *Combinatorics, Probability and Computing*, pages 577–625, 2004.